

Cenário

Vamos criar um sistema de autenticação onde diferentes etapas verificam:

1. Se o usuário existe.
2. Se a senha está correta.
3. Se o usuário tem permissão para acessar o recurso.

Cada etapa será um **handler** na cadeia de responsabilidade.

Implementação

1. Criar a Interface Base do Handler

Define a estrutura comum para todos os manipuladores.

```
public interface IHandler
{
    void SetNext(IHandler next);
    void Handle(Request request);
}
```

2. Criar a Classe Base para os Handlers

Facilita a implementação da lógica para passar o pedido adiante.

```
public abstract class BaseHandler : IHandler
{
    private IHandler _nextHandler;

    public void SetNext(IHandler next)
    {
        _nextHandler = next;
    }

    public virtual void Handle(Request request)
    {
        if (_nextHandler != null)
        {
            _nextHandler.Handle(request);
        }
    }
}
```

3. Criar os Handlers Concretos

Cada classe executa uma verificação específica e, se necessário, passa o pedido para o próximo handler.

```
public class UserExistsHandler : BaseHandler
{
    public override void Handle(Request request)
    {
        if (request.Username != "admin")
        {
            Console.WriteLine("User does not exist.");
            return;
        }

        Console.WriteLine("User exists.");
        base.Handle(request);
    }
}

public class PasswordHandler : BaseHandler
{
    public override void Handle(Request request)
    {
        if (request.Password != "1234")
        {
            Console.WriteLine("Incorrect password.");
            return;
        }

        Console.WriteLine("Password is correct.");
        base.Handle(request);
    }
}

public class PermissionHandler : BaseHandler
{
    public override void Handle(Request request)
    {
        if (!request.HasPermission)
        {
            Console.WriteLine("Permission denied.");
            return;
        }

        Console.WriteLine("Permission granted.");
        base.Handle(request);
    }
}
```

4. Criar o Objeto de Pedido

O pedido que será processado pela cadeia.

```
public class Request
{
    public string Username { get; set; }
    public string Password { get; set; }
    public bool HasPermission { get; set; }
}
```

5. Configurar e Usar a Cadeia de Responsabilidade

```
class Program
{
    static void Main()
    {
        // Criar os handlers
        var userExistsHandler = new UserExistsHandler();
        var passwordHandler = new PasswordHandler();
        var permissionHandler = new PermissionHandler();

        // Configurar a cadeia
        userExistsHandler.SetNext(passwordHandler);
        passwordHandler.SetNext(permissionHandler);

        // Criar o pedido
        var request = new Request
        {
            Username = "admin",
            Password = "1234",
            HasPermission = true
        };

        // Processar o pedido
        userExistsHandler.Handle(request);

        // Saída esperada:
        // User exists.
        // Password is correct.
        // Permission granted.
    }
}
```

Vantagens do Chain of Responsibility

1. **Desacoplamento:** O remetente do pedido não precisa saber qual handler processará o pedido.
2. **Flexibilidade:** Fácil adicionar, remover ou reorganizar handlers na cadeia.
3. **Responsabilidade distribuída:** Cada handler é responsável por uma etapa específica.

Quando evitar?

- Se o pedido precisa ser processado por todos os handlers (use um padrão diferente, como **Composite**).
- Se a lógica for simples e um único método puder lidar com o pedido.

Resumo

O **Chain of Responsibility** é uma ótima escolha quando você precisa que um pedido passe por várias etapas sequenciais de processamento, permitindo que cada etapa decida se deve manipular o pedido ou encaminhá-lo. □